



AIoT and Cyber Security TAG, Amrita School of Computing, Bangalore

Student Preparation Guide

Everything you need to build real-world-ready security solutions

CYBER SECURITY END SEM HACKATHON



6 Hours
Sprint Format



Attack / Defend
Choose Your Track



Industry-Ready
Real Architectures

*Problem statements are revealed on the day of the hackathon.
Use this guide to prepare your thinking, tooling, and architecture awareness.*

1. Hackathon Overview

1.1 What This Hackathon Is About

This hackathon challenges you to design, build, and demonstrate a cybersecurity solution within a 6-hour window. Problem statements spanning both offensive (attack/red-team) and defensive (blue-team/architecture) themes will be handed out on the day. You will not know the exact problem statement in advance — but you can and should prepare.

The core expectation is not just that your solution works in isolation. It must be realistic: your solution should fit into the security architecture of a real organisation without requiring a complete teardown of existing infrastructure.

1.2 The Two Tracks

⌘ Offensive / Red-Team Track	🛡 Defensive / Blue-Team Track
Discover, exploit, and demonstrate vulnerabilities in a given system or environment.	Detect, mitigate, and respond to security threats across an architectural environment.
Deliverable: A working proof-of-concept exploit, attack chain, or penetration report.	Deliverable: A working detection/prevention tool, policy enforcement layer, or monitoring solution.
Mindset: "How would an adversary break this?"	Mindset: "How do I make breaking this harder, and how do I know when someone tries?"
Examples: SSRF chaining, privilege escalation, credential harvesting, lateral movement.	Examples: SIEM rule creation, WAF tuning, zero-trust policy, anomaly detection dashboard.

1.3 Day-of Schedule (6-Hour Sprint)

0:00 – 0:30	Problem Statement Reveal + Q&A with organisers
0:30 – 1:00	Architecture scoping — decide what env your solution targets
1:00 – 4:30	Core build phase — implement, iterate, and test your solution
4:30 – 5:15	Integration check — verify your solution works in target architecture context
5:15 – 5:45	Documentation and demo preparation
5:45 – 6:00	Submission and short live demo

📌 Important Note on Problem Statements

Problem statements are deliberately revealed on the day to simulate real incident response and triage conditions.

Each statement will be tagged with one or more target architectures (see Section 2) so you know the environment context.

Read the problem statement at least twice before writing a single line of code.

Spend the first 30 minutes understanding the threat model — not coding.

2. Real-World Company Architectures

Most security solutions fail at adoption because they are built in a vacuum. The following sections describe the dominant infrastructure architectures used by companies today. When your problem statement is revealed, identify which architecture type(s) it targets — then design your solution to fit naturally inside that environment.

2.1 Monolithic On-Premises (Traditional Enterprise)

What It Looks Like

A single large application or a set of tightly coupled services running on physical or virtualised servers inside the organisation's own data centre. There is a well-defined network perimeter protected by a hardware firewall, VPN gateway, and DMZ.

Typical Technology Stack

- Active Directory / LDAP for identity and access management
- Physical or VMware/Hyper-V virtualised servers
- Cisco / Palo Alto / Fortinet perimeter firewalls
- Centralised logging (Syslog, Windows Event Log forwarded to SIEM)
- NFS/SMB shared file systems, Oracle/MSSQL databases on-prem

Attack Surface to Keep in Mind

- Credential attacks against Active Directory (Pass-the-Hash, Kerberoasting, DCSync)
- Lateral movement via SMB, RDP, WMI within the flat internal network
- Exploitation of unpatched software (patch cycles are often slow on-prem)
- Insider threats — privileged users have broad access

Defensive Constraints

- No direct internet egress from servers; solutions cannot rely on cloud APIs
- Agent deployment is slow — prefer agentless or lightweight network-based detection
- Must integrate with existing SIEM (Splunk, IBM QRadar, ArcSight) via syslog/CEF
- Changes require change-management approval — your solution must be non-disruptive to deploy

2.2 Cloud-Native / Microservices Architecture

What It Looks Like

Applications decomposed into dozens (sometimes hundreds) of independently deployed services, each running in containers (Docker) orchestrated by Kubernetes. Hosted on a hyperscaler: AWS, Azure, or GCP. No traditional network perimeter — security is implemented at the service layer.

Typical Technology Stack

- Kubernetes (EKS, AKS, GKE) for container orchestration
- Service mesh: Istio or Linkerd for mTLS between services
- Cloud IAM (AWS IAM, Azure AD, GCP IAM) — identity is the new perimeter
- Cloud-native logging: AWS CloudTrail, Azure Monitor, GCP Cloud Audit Logs
- Secrets management: HashiCorp Vault, AWS Secrets Manager
- API Gateway (Kong, AWS API Gateway) as the single ingress point

Attack Surface to Keep in Mind

- Container escape — breakout from a compromised pod to the node
- IMDS (Instance Metadata Service) abuse for credential theft
- Overly permissive IAM roles (privilege escalation via assume-role)
- Supply chain attacks — malicious base images or dependencies
- SSRF targeting internal cloud metadata endpoints

Defensive Constraints

- Solutions must work with ephemeral infrastructure — containers restart constantly
- Deploy as a Kubernetes admission webhook, DaemonSet, or sidecar — not a separate VM
- Must not break horizontal scaling — stateless by design
- Integrate with cloud-native alerting (SNS, PagerDuty, Slack webhook)

2.3 Hybrid Architecture (On-Prem + Cloud)

What It Looks Like

The most common architecture in large enterprises today. Core databases and legacy applications remain on-prem while new workloads, SaaS integrations, and development environments live in the cloud. Connected via VPN tunnels, AWS Direct Connect, or Azure ExpressRoute.

Typical Technology Stack

- Azure AD Connect / AWS SSO bridging on-prem AD to cloud identity
- Hybrid DNS: internal DNS servers + Route 53 or Azure DNS
- Data flows in both directions — cloud apps read from on-prem databases
- Microsoft Defender for Endpoint / CrowdStrike across both environments
- SIEM: Microsoft Sentinel or Splunk ingesting from both cloud and on-prem

Attack Surface to Keep in Mind

- The identity bridge (AD Connect, SSO) is a critical choke point — compromise it and you own both environments
- Lateral movement from cloud to on-prem via VPN trust relationships
- Shadow IT — developers spin up cloud resources that bypass on-prem controls
- Split-brain DNS attacks confusing internal vs. external resolution

Defensive Constraints

- Your solution must monitor activity across both environments from a single pane
- Avoid creating new trust paths that themselves become attack vectors
- Log normalisation is a real challenge — different schemas on each side

2.4 Zero-Trust Architecture (ZTA)

What It Looks Like

"Never trust, always verify." Every access request — regardless of source — is authenticated, authorised, and continuously validated. There is no implicit trust for internal traffic. Often layered on top of one of the architectures above.

Typical Technology Stack

- Identity Provider (Okta, Azure AD, Ping Identity) with MFA enforced everywhere
- BeyondCorp / Google IAP / Cloudflare Access for application-layer proxying
- Device posture checks before granting access (MDM: Intune, Jamf)
- Micro-segmentation: east-west traffic policies via NSX or Illumio
- Continuous authorisation: UEBA (User and Entity Behaviour Analytics)

Attack Surface to Keep in Mind

- IdP compromise is catastrophic — all access flows through it
- Token theft / session hijacking bypasses device posture checks
- Over-broad OAuth consent grants (third-party apps with excessive permissions)
- Policy misconfiguration — overly permissive rules that defeat the ZT model

Defensive Constraints

- Every component must log identity context (who, from what device, what resource)
- Solutions must work with existing IdP APIs — do not re-implement authentication
- Policy changes must be declarative and auditable (GitOps-style configuration)

2.5 OT/IoT & Critical Infrastructure

What It Looks Like

Operational Technology (OT) environments include industrial control systems (ICS), SCADA systems, PLCs, and IoT sensors running in manufacturing, energy, utilities, or healthcare. These systems have strict uptime requirements and often run legacy protocols.

Typical Technology Stack

- Protocols: Modbus, DNP3, OPC-UA, MQTT, BACnet — not TCP/IP-native
- Air-gapped or semi-air-gapped network segments
- Historian servers (OSIsoft PI) bridging OT and IT networks
- HMI (Human-Machine Interfaces) running Windows XP/7 — unpatched by design
- Network monitoring via passive sensors (Claroty, Dragos, Nozomi)

Attack Surface to Keep in Mind

- IT/OT boundary — the historian or jump server is the weak link
- Unauthenticated industrial protocols — Modbus has no auth by design
- USB-borne malware (Stuxnet model) bypassing network controls
- Ransomware that shuts down physical processes (Colonial Pipeline model)

Defensive Constraints

- NEVER disrupt live OT traffic — passive monitoring only in most cases
- Cannot deploy agents on PLCs or SCADA — network-level visibility only
- Solutions must account for long patch cycles (years, not weeks)
- Alerting must reach human operators in real time — not just log platforms

3. How to Think About Your Project

The difference between a hackathon project that impresses and one that falls flat is almost always this: did the team treat the problem like a real engineering challenge inside a real environment, or did they build a tool that works only in their demo VM?

3.1 The Five Questions to Answer Before Writing Code

#	Question	Why It Matters
1	What architecture does this problem live in?	Determines your constraints and the tools already available to you.
2	Who is the adversary / what is the threat?	Shapes what you build — DDoS defence is different from insider threat detection.
3	Where does your solution sit in the stack?	Network? Host? Application? Identity layer? Wrong layer = wrong tool.
4	What does success look like operationally?	An alert, a blocked request, a forensic report? Define it first.
5	How would this be deployed without breaking things?	Shows you understand change-management and production risk.

3.2 Choosing Your Layer of Intervention

Every security solution operates at a specific layer of the stack. Choosing the wrong layer is the most common mistake. Use this model:

Layer	What You Control	Typical Tooling
Network / Perimeter	Packet flows, firewall rules, BGP routes	Snort, Suricata, pfSense, Zeek
Host / Endpoint	Processes, files, registry, kernel calls	OSSEC, Wazuh, CrowdStrike, EDR agents
Application / API	HTTP requests, JWT tokens, input validation	WAF, RASP, OWASP ModSecurity
Identity / IAM	Auth flows, role bindings, OAuth scopes	Okta, Azure AD Conditional Access, PAM
Data / Storage	Encryption at rest, data classification, DLP	Macie, Varonis, PGP, TLS 1.3
Cloud Control Plane	Resource provisioning, IAM policies, audit logs	AWS Config, Prisma Cloud, Checkov

3.3 Feasibility Checklist — Before You Demo

Before presenting your solution, verify these points. They are the same questions a company's security or infrastructure team will ask before adopting any tool.

☑ Pre-Demo Feasibility Checklist

- ☐ Does your solution require root / admin on every target host, or can it run with least privilege?
- ☐ Can it be deployed incrementally — one server, one pod, one region at a time?
- ☐ Does it produce false positives at a rate that is operationally acceptable?
- ☐ Will it still work if the attacker knows the tool is present (security by obscurity is not a strategy)?
- ☐ Does it integrate with at least one standard log/alert output (syslog, SIEM CEF, webhook)?
- ☐ Does it have a clearly defined off switch or rollback procedure?
- ☐ Does it comply with data residency constraints — does it exfiltrate logs to external services?

3.4 The MITRE ATT&CK Framework — Your Universal Language

Whether you are attacking or defending, anchor your work to the MITRE ATT&CK framework. It is the shared vocabulary used by red teams, blue teams, SOC analysts, and security product vendors across the industry.

If you are on the Offensive Track...	If you are on the Defensive Track...
Identify which ATT&CK Tactics your attack chain covers (e.g., Initial Access → Execution → Lateral Movement).	Identify which ATT&CK Techniques your defence detects or prevents.
Map your exploit steps to specific ATT&CK Technique IDs (e.g., T1055 — Process Injection).	Map your detection rules/signatures to ATT&CK Technique IDs.
Demonstrate the full kill chain — not just a single exploit.	Show that your defence does not just block one technique but a class of techniques.
Document mitigations your team would recommend after the attack.	Document which techniques you do NOT yet cover — honest gap analysis impresses.

3.5 Scoping Your Solution for 6 Hours

Six hours is short. Most teams over-scope and under-deliver. Use this prioritisation model:

Priority	Focus	Time Allocation
P0 — Core	The single scenario your solution handles end-to-end, demonstrably and reliably.	~3.5 hrs
P1 — Extension	A second related scenario or a configuration option that widens applicability.	~1.5 hrs
P2 — Polish	Logging, alerting, a simple UI, or documentation that makes deployment realistic.	~1 hr

4. Track-Specific Guidance

4.1 Offensive Track — Building an Ethical Attack Demonstration

Ground Rules

- All offensive work must be performed only on infrastructure provided or explicitly authorised for the hackathon.
- Demonstrate impact — do not cause actual damage to systems, data, or services.
- Document your steps as you go; a well-written attack report is as important as the exploit itself.
- Think like a threat actor, report like a consultant.

Structure of a Strong Offensive Submission

- Reconnaissance: What information did you gather? How would a real attacker do this?
- Initial Access: What vulnerability or misconfiguration did you exploit?
- Execution / Persistence: How did you establish a foothold?
- Lateral Movement / Privilege Escalation: How did you expand access?
- Impact Demonstration: What could a real attacker do at this point?
- Recommended Mitigations: What fixes would close the attack path?

⚠ Offensive Track — Things to Avoid

Do not run vulnerability scanners on any system outside the designated hackathon environment.

Do not attempt to access real credentials, real databases, or real user data.

Do not deploy any tool that persists after the hackathon environment is torn down.

Do not attempt denial-of-service attacks — demonstrate the vulnerability another way.

4.2 Defensive Track — Building a Production-Credible Solution

What Separates Good Defensive Solutions

- Detection with context: Not just "anomaly detected" but "this matches T1078 — Valid Accounts abuse via this specific IAM policy"
- Low false-positive rate: A tool that fires 500 alerts per hour will be turned off on day one
- Actionable output: Every alert should suggest a next step for the analyst
- Graceful degradation: If your solution goes down, it should fail open (continue allowing traffic) not fail closed (block everything)

Defensive Solution Architecture Template

A robust defensive solution should have three logical components:

Component	Purpose	Example Implementation
Sensor / Collector	Gathers raw data from the environment	Network tap, syslog forwarder, K8s audit log webhook, API event stream
Analyser / Detector	Applies logic to identify threats	Rule engine, ML model, signature matching, threshold-based alerting
Responder / Reporter	Takes action or notifies humans	Firewall rule push, Slack alert, ticket creation, dashboard widget

5. Recommended Tools & Technologies

The following tools are widely used in professional security work. Familiarise yourself with at least one tool in each category relevant to your track before the hackathon day.

5.1 Offensive / Red-Team Tooling

Tool / Framework	Primary Use Case
Metasploit Framework	Exploit development, payload generation, post-exploitation modules
Burp Suite (Community)	Web application testing — intercepting proxy, scanner, repeater
Nmap / Masscan	Network discovery and port scanning — always scope-limited
Gobuster / ffuf	Directory/file brute-forcing and subdomain enumeration
Impacket (Python)	SMB, LDAP, Kerberos protocol interaction for AD attacks
BloodHound / SharpHound	Active Directory attack path mapping and privilege escalation visualisation
Responder	LLMNR/NBT-NS poisoning for credential capture on LAN
SQLMap	Automated SQL injection detection and exploitation
Nuclei	Template-based vulnerability scanning (FOSS, used by professionals)
CyberChef	Data transformation, encoding/decoding, forensic analysis

5.2 Defensive / Blue-Team Tooling

Tool / Framework	Primary Use Case
Wazuh (open-source SIEM/EDR)	Host-based IDS, log analysis, compliance checking
Suricata / Snort	Network-based IDS/IPS with rule-based detection
Zeek (formerly Bro)	Passive network analysis and rich protocol metadata extraction
The Elastic Stack (ELK)	Log ingestion, indexing, and dashboarding (widely used in SOCs)
Sigma Rules	SIEM-agnostic detection rule language — write once, deploy anywhere
OpenVAS / Greenbone	Vulnerability assessment of your own infrastructure
Falco	Cloud-native runtime threat detection for Kubernetes/containers
YARA	Pattern-based malware classification and file scanning
Velociraptor	Endpoint forensics and live response — deployed via single binary

Tool / Framework	Primary Use Case
Atomic Red Team	Library of adversary simulation tests mapped to MITRE ATT&CK

5.3 Architecture & Development Tools

Tool / Technology	Why It Matters for This Hackathon
Docker + Docker Compose	Quickly spin up isolated lab environments for testing without modifying your host
Wireshark / tcpdump	Packet capture and protocol analysis — essential for both tracks
Python (Scapy, Requests, Paramiko)	Rapid tool development, protocol scripting, automation
Terraform (basics)	Shows you understand infrastructure-as-code and can describe your deployment
Git + GitHub	Version-control your code throughout the hackathon — expected by industry
Markdown / README	A clear README is part of a professional submission

6. Mapping Your Solution to Company Architecture

This is the most important intellectual exercise of the hackathon. When you receive your problem statement, use the following framework to map your solution to the target architecture.

6.1 Architecture Fit Matrix

For each possible solution approach, ask whether it fits the constraints of the target architecture. Use this matrix as a quick reference:

Solution Approach	On-Prem	Cloud-Native	Hybrid	Zero-Trust	OT/IoT
Network-level passive sensor	✓	⚠	✓	⚠	✓
Host-based agent	✓	⚠	✓	✓	✗
Cloud API integration	✗	✓	✓	✓	✗
Kubernetes sidecar/webhook	✗	✓	⚠	⚠	✗
SIEM rule / Sigma rule	✓	✓	✓	✓	⚠
Identity/IdP integration	✓	✓	✓	✓	✗
Firewall rule automation	✓	⚠	✓	✗	⚠

✓ = Generally fits ⚠ = Fits with conditions/limitations ✗ = Typically not applicable

6.2 Deployment Narrative — What to Include in Your Demo

Your demo must answer the question: "If we hired you to deploy this tomorrow, what would that look like?" Prepare a 2-minute deployment narrative covering:

- Pre-requisites: What does the environment need before your tool can run?
- Deployment steps: How is it installed? (Agent, container, API registration, policy upload?)
- Configuration: What does an operator tune? What are the defaults?
- Observability: How does an operator know it is working?
- Day-2 operations: How is it updated? How are false positives managed?

6.3 Avoiding Common Architecture Anti-Patterns

ⓘ Anti-Patterns That Will Get Your Solution Rejected in Industry

"It just needs root on every server." — Privilege minimisation is non-negotiable in production.

"It sends all logs to our cloud endpoint." — Data residency and privacy laws forbid this in many industries.

"You have to disable the existing AV first." — Solutions that require removing existing controls will never be approved.

"It works best if you turn off TLS inspection." — Security solutions cannot require weakening security posture.

"We haven't tested it under load yet." — Performance impact must be characterised before deployment.

"The detection rules are hardcoded." — Operators need to tune thresholds for their environment.

7. How to Prepare Before Hackathon Day

7.1 Knowledge Areas to Review

Topic	Why It Will Help You on the Day
OSI model and common protocols (TCP/IP, DNS, HTTP/S, TLS)	Almost every problem statement will involve network traffic at some layer
Active Directory fundamentals (Kerberos, LDAP, GPO)	AD attacks appear in ~60% of enterprise security scenarios
Linux command line and bash scripting	Most tooling runs on Linux; scripting saves hours during the sprint
Docker basics (run, exec, logs, network)	Most lab environments and deployment targets are containerised
Basic Python (socket, requests, subprocess libraries)	Fastest language for rapid security tool prototyping
OWASP Top 10 (for web-focused problems)	Web vulnerabilities remain the most common initial access vector
Cloud IAM concepts (roles, policies, assume-role, service accounts)	Cloud problems are increasingly common
Log formats: CEF, JSON structured logging, Windows Event IDs	You will read a lot of logs, and you need to parse them quickly

7.2 Setting Up Your Environment in Advance

Do not spend hackathon time installing tools. Set up the following before the day:

- A Linux VM (Kali Linux, Parrot OS, or Ubuntu) with networking configured
- Docker and Docker Compose installed and tested with a hello-world container
- Python 3.x with pip; install: requests, scapy, impacket, paramiko, flask
- A local Git repository with your team's shared workspace and a good .gitignore
- At least one note-taking system agreed upon as a team (shared doc, Notion, etc.)
- Wireshark and tcpdump installed and verified capturing on your primary interface
- Burp Suite Community Edition installed and browser proxy configured

7.3 Team Roles to Assign in Advance

Role	Responsibilities During the Hackathon
Lead Architect	Owns the solution design decision; makes the call when the team disagrees on approach
Primary Builder	Writes the core code/configuration; uninterrupted coding time is their priority
Research & Integration	Looks up documentation, finds edge cases, sets up environment and dependencies
Documentation & Demo Lead	Writes the README, prepares the demo script, owns the presentation narrative

7.4 Suggested Resources

- MITRE ATT&CK Navigator: attack.mitre.org — bookmark it now
- HackTricks (book.hacktricks.xyz) — comprehensive attack technique reference
- TryHackMe / HackTheBox — practice environments in the weeks before the event
- OWASP Testing Guide (owasp.org/www-project-web-security-testing-guide)
- Sigma HQ GitHub (github.com/SigmaHQ/sigma) — detection rule library
- Cloud Security Alliance (cloudsecurityalliance.org) — cloud architecture security guidance
- NIST SP 800-207: Zero Trust Architecture (free PDF from nist.gov)

8. What to Submit at the End

8.1 Required Deliverables

Every team must submit the following by the end of hour 6:

- Working demonstration: A live, runnable prototype — not slides alone
- Source code: In a version-controlled repository (GitHub/GitLab) with a clear README
- Architecture diagram: One diagram showing where your solution sits in the target architecture
- Threat model document: One page describing the threat your solution addresses and the MITRE ATT&CK mapping
- Deployment brief: Half a page describing how this would be deployed in a real company environment

8.2 What Makes a Strong README

Your README is often the first thing evaluated. It should answer:

- What problem does this solve? (One paragraph — clear and direct)
- Which architecture does it target? (On-prem, cloud-native, hybrid, ZT, OT)
- What MITRE ATT&CK techniques does it address?
- How do you run it? (Step-by-step, copy-paste ready)
- What dependencies does it have?
- What are known limitations or gaps?

8.3 The 5-Minute Demo Structure

Time	What to Cover
0:00 – 1:00	Context: What is the threat? Which company architecture is affected? What does failure look like without your solution?
1:00 – 3:00	Live demo: Show the attack or the detection/prevention in action. Use real terminal output — not screenshots.
3:00 – 4:00	Architecture fit: Show your diagram. Explain where the tool sits and how it integrates with existing infrastructure.
4:00 – 5:00	Limitations and next steps: What did you not build? What would the full production version add? This shows intellectual honesty.

9. Ethics, Conduct, and Legal Boundaries

9.1 Ethical Boundaries — Non-Negotiable

🛡️ Legal and Ethical Conduct Requirements

All offensive activity is strictly limited to the designated hackathon lab environment.

Scanning, probing, or attacking any systems outside the provided environment is illegal under the Computer Fraud and Abuse Act (CFAA), IT Act (India), and equivalent laws worldwide.

Do not exfiltrate any real data — simulated data only.

Do not plant persistent backdoors or malware of any kind, even in the lab.

Report any unexpected access to systems you were not intended to access immediately to the organisers.

Responsible disclosure principles apply: document vulnerabilities, do not exploit for personal gain.

9.2 The Security Professional Mindset

Every capability you build in this hackathon is dual-use. The same Python script that discovers open ports for a defender also works for an attacker. The discipline of the security professional is context, consent, and documentation.

- Context: You know what environment you are authorised to operate in.
- Consent: You have explicit permission to test what you are testing.
- Documentation: You record what you did, when, and why — always.

These three principles are what separate a security engineer from a threat actor. Carry them into everything you build today and in your career.

9.3 Handling Discovered Vulnerabilities Responsibly

If during your work you discover a vulnerability in a real system (not part of the hackathon lab), the correct response is:

- Stop. Do not exploit further.
- Document what you found: URL, parameter, reproduction steps.
- Report it to the system owner via responsible disclosure channels (security@domain or HackerOne if they have a bug bounty).
- Allow reasonable time for a fix before any public disclosure.
- Do not share, sell, or publish exploit code for unpatched vulnerabilities.

10. Quick Reference Cards

10.1 Architecture Decision Tree

When your problem statement arrives, work through this tree in under 10 minutes:

If the problem involves...	Target Architecture is likely...
Active Directory, domain controllers, GPOs, SMB shares	On-Premises / Hybrid
Kubernetes, Docker, ECR/GCR, cloud IAM roles, S3/GCS buckets	Cloud-Native
VPN tunnels, ExpressRoute, AD Connect, hybrid identity	Hybrid
Zero-trust proxy, device posture, MFA enforcement, UEBA	Zero-Trust Architecture
PLCs, SCADA, Modbus, DNP3, historian servers, HMIs	OT / IoT / Critical Infrastructure
Web app vulnerabilities, JWT, OAuth, API endpoints	Cloud-Native or Hybrid (Application Layer)
Phishing, email security, credential harvesting	Any — focus on Identity layer

10.2 The 30-Minute Triage Checklist

Use this immediately after the problem statement is revealed:

- Read the problem statement twice before speaking
- Identify the target architecture (use the decision tree above)
- Identify the attack/threat category and map it to MITRE ATT&CK
- Decide your solution layer: Network? Host? Application? Identity? Data? Cloud control plane?
- Agree on your P0 (core deliverable) as a team — one sentence
- Assign roles: who builds, who researches, who documents, who demos
- Write your first three implementation steps — then start coding

10.3 Useful Command Snippets

Purpose	Command
Quick port scan	<code>nmap -sV -sC -T4 <target-ip> -oN scan.txt</code>
Capture packets	<code>tcpdump -i eth0 -w capture.pcap</code>
List Docker containers	<code>docker ps -a && docker logs <container></code>
Check open connections	<code>ss -tlnp or netstat -tlnp</code>
View system logs (Linux)	<code>journalctl -xe -n 100 or tail -f /var/log/syslog</code>
Find SUID binaries	<code>find / -perm -4000 -type f 2>/dev/null</code>
Extract strings from binary	<code>strings <binary> grep -iE 'pass key token secret'</code>

Purpose	Command
Spin up quick HTTP server	<code>python3 -m http.server 8080</code>
Decode base64	<code>echo '<data>' base64 -d</code>
Check AWS caller identity	<code>aws sts get-caller-identity</code>

Good luck — think like the adversary, build like an engineer, present like a consultant.

*The best security solutions are not the most complex ones.
They are the ones that actually get deployed.*